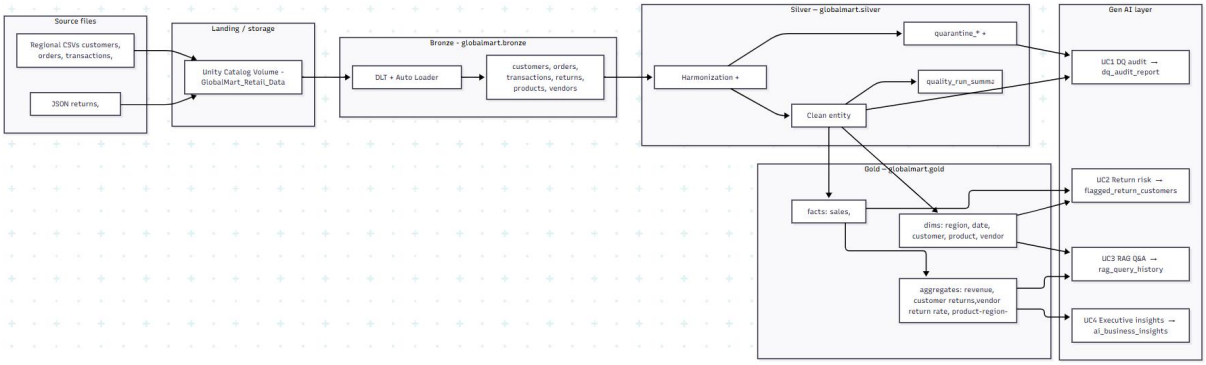
GlobalMart - Architecture Document

Medallion pipelines (`Bronze.py`, `Silver.py`, `Gold.py`) + Gen AI notebooks (UC1–UC4).

1. Data flow diagram (end-to-end)



Narrative (source → user): Regional CSV and JSON files land in a **Volume**. **Bronze** ingests them incrementally (Auto Loader), preserving lineage. **Silver** standardizes schemas, applies **data quality rules**, and splits **pass** vs **quarantine** rows. **Gold** builds a **star schema** (dimensions + facts) and **pre-aggregates** KPIs. **Gen AI** notebooks read **Silver quarantine** and/or **Gold** tables and write **Gold Delta tables** with narratives or scores. **Finance, returns, and merchandising** consume results via **Unity Catalog**, **SQL**, and **dashboards**.

2. Component map

Component	Layer	Role
CSV / JSON source files (`GlobalMart_Retail_Data/`)	Source	Authoritative retail extracts (multi-region).
Unity Catalog Volume (`/Volumes/globalmart/raw/landing_files/...`)	Landing	Stores raw files for Auto Loader paths in `Bronze.py`.
Delta Live Tables (DLT) — Python	Pipeline orchestration	Declarative tables/views; Bronze + Silver + Gold pipelines.
Auto Loader (`cloudFiles`)	Bronze	Streaming ingestion; schema hints + evolution at `_schemas/dlt_bronze`.
Apache Spark / PySpark	Bronze–Gold	Transformations, joins, windowing, aggregates.

Component	Layer	Role
Delta Lake	Bronze–Gold + AI outputs	ACID tables, time travel, `saveAsTable` / DLT targets.
Unity Catalog (`globalmart` catalog, `bronze` / `silver` / `gold` schemas)	Governance	Table discovery, ACLs, lineage to consumers.
Databricks Notebooks	Development / Gen AI	UC1–UC4: LLM calls, RAG, `ai_query()` demos.
OpenAI-compatible client (`openai` SDK, Databricks token)	Gen AI	Calls serving endpoint **`databricks-gpt-oss-20b`**.
Sentence Transformers + FAISS (UC3)	Gen AI	Local embeddings + vector search for RAG.
`ai_query()` (UC4 SQL cells)	Gen AI	SQL-side LLM prompts on KPI JSON.

3. Data quality strategy (by layer)

Bronze

- **Ingest as-is** with **`_ingest_timestamp`** and **`_source_file_path`** for lineage.
- **Schema discipline:** `inferSchema=false` + **Auto Loader schema location** + **`addNewColumns`** evolution.
- **Format handling:** CSV with header; JSON multi-line; recursive file lookup for regional paths.

Silver

- **Harmonization:** One canonical id per entity (e.g. coalesce regional `customer_id` variants); timestamp and money parsing; segment/region/return-status normalization; **Region 4 city/state fix** using file path.
- **Row-level rules** encoded in **`failure_reasons`** (semicolon-separated tokens), e.g. null keys, invalid vendor pattern, non-US country (customers), negative sales (transactions), invalid return status (returns), etc.
- **Split outcomes:** Valid rows → **`globalmart.silver.<entity>`**; invalid → **`globalmart.silver.quarantine_<entity>`** (retains `failure_reasons`).
- **DLT expectations (`@dlt.expect`)** on silver tables for critical columns (e.g. non-null ids, vendor `VEN##` pattern on orders).
- **Monitoring: `globalmart.silver.quality_run_summary`** — accepted vs rejected counts per entity per run.

Gold

- **Consume only trusted Silver** pass tables (no quarantine in facts/dims).
- **Referential consistency** enforced by join design (e.g. `fact_sales` inner-joins orders + customers).
- **Business definitions** fixed in facts/aggregates (grain documented in `Gold.py` comments).

4. Analytics layer structure (Gold — top of stack)

Catalog: `globalmart.gold`

Dimensions (descriptive context)

Table	What it measures / holds	Keys / notes
dim_region	Distinct **regions** from customers	`region_key`, `region_name`
dim_date	**Calendar** attributes for order and return dates	`date_key` (YYYYMMDD), year, quarter, month, `year_month`
dim_customer	**Customer** attributes: segment, region, address	`customer_id` → links to facts
dim_product	**Product catalog** (from Silver products)	`product_id` → links to `fact_sales`
dim_vendor	**Vendor** reference	`vendor_id` → links to facts

Facts (events / line-level measures)

Table	Grain	What it measures
fact_sales	One row per **order line** (order + product)	`sales_line_key`, `order_id`, `product_id`, `customer_id`, `vendor_id`, `region_key`, `date_key`, `year_month`, **sales_amount**, **quantity**, **profit**, **discount_rate**, **discount_amount**, flags

Table	Grain	What it measures
fact_return	One row per **return**	`return_key`, `order_id`, **refund_amount**, **return_date** / `return_date_key`, `return_status`, `vendor_id`, `customer_id`, `region_key`

Aggregates (KPIs for dashboards and AI)

Table	Grain	What it measures
agg_monthly_revenue_by_region	**Month × region**	`total_sales`, `total_profit`, `order_count`, `total_quantity`
agg_customer_return_summary	**Customer × region**	`return_count`, `total_refund_amount`, `avg_refund_amount`
agg_vendor_return_rate	**Vendor**	`vendor_gross_sales`, `vendor_refunds`, **`refund_to_sales_ratio`**
agg_product_region_monthly	**Product × region × month**	`total_sales`, `total_quantity`, `total_profit`, `order_count`

How they relate

- **Facts** connect to **dimensions** via **`customer_id`**, **`product_id`**, **`vendor_id`**, **`region_key`**, **`date_key`** / **`year_month`**.
- **agg_*** tables roll up **fact_sales** and/or **fact_return** (and join **dim_region** or **dim_vendor** where names are needed).
- **Star schema pattern:** filter/slice on dimensions and aggregates; drill to **facts** for detail.

5. Generative AI integration

Position: Sits **above Gold** (and selectively **Silver quarantine** for DQ detail), **writes back** to **Gold Delta tables** or logs; does **not** train on raw Bronze.

Use case	Notebook	Inputs (what goes in)	Outputs (what comes out)

Use case	Notebook	Inputs (what goes in)	Outputs (what comes out)
UC1 — AI Data Quality Reporter	`UC1 Designing the AI Data Quality Reporter.ipynb`	**`globalmart.silver.quarantine_*`** (`failure_reasons` exploded by issue), mapping **issue_type → field**; optional alignment with **`quality_run_summary`**	**`globalmart.gold.dq_audit_report`** — per-entity, per-issue **plain-English audit narratives** + counts
UC2 — Returns fraud / risk	`UC2 Engineering the Returns Fraud Investigator.ipynb`	**`fact_return`**, **`dim_customer`**, **`fact_sales`** — rule-based **anomaly_score**, thresholds	**`globalmart.gold.flagged_return_customers`** — risk scores, **rules violated**, **AI investigation brief**
UC3 — Product / vendor Q&A (RAG)	`UC3 Building the Product Intelligence Assistant.ipynb`	**`dim_product`**, **`dim_vendor`**, **`agg_product_region_monthly`**, **`agg_vendor_return_rate`** (enriched **documents**)	**`globalmart.gold.rag_query_history`** — question, **grounded answer**, retrieved doc metadata
UC4 — Executive BI	`UC4 Synthesizing Executive Business Intelligence.ipynb`	**KPI JSON** from **`agg_monthly_revenue_by_region`**, **`agg_vendor_return_rate`**, **`agg_product_region_monthly`** (no raw-row prompts)	**`globalmart.gold.ai_business_insights`** — **insight_type**, **executive_summary**, `kpi_payload_json`; plus **`ai_query()`** demos in SQL

Model: `databricks-gpt-oss-20b` via Databricks **serving endpoints** (OpenAI-compatible API). UC3 adds **sentence-transformers** + **FAISS** for retrieval.